A stabilizer-rank residual backend for `clifft`: an honest assessment on magic-sparse circuits

Research note (revised)

July 9, 2026

Abstract

`clifft` simulates Clifford+T circuits with a symbolic Clifford frame plus a dense 2^k active block. We build and validate a low-rank stabilizer (CH-form) backend for the magic-sparse regime: minimal-extent $\{I, S\}$ magic decomposition ($2^{0.228t}$), single-shot tensor-product magic sampling, BGH norm estimation, and a global Clifford-frame composition, each checked against a dense oracle and against `clifft` to machine precision. This revision replaces the earlier draft's headline claim, which an adversarial review refuted: the "crossover at $n \approx 22$" was measured against `clifft`'s *unitary* path (peak rank n), while `clifft`'s own `record_probabilities` answers the same queries *exactly* from the measured program — at peak rank 1 on that benchmark's chain-IQP family ($\sim 10^5\times$ faster than the number we charged it), and at peak rank $n/2$ on dense random IQP. Re-benchmarked fairly (dense IQP, exact $2^{n/2}$ baseline, exact meet-in-the-middle ground truth, and an error metric that charges the *full* pipeline), the true picture is: `clifft`'s exact fast path wins to $n \approx 34$; the backend overtakes at $n \approx 34$ for TV ≈ 0.17 and at $n \approx 48$ for TV ≈ 0.08, with a clean measured accuracy dial TV $= (0.50 \pm 0.01)\,\delta$; and beyond $n \approx 62$ — where every exact method (dense block, forced replay, meet-in-the-middle) needs $\gtrsim$16 GB–1 TB — the backend is the only method that runs (n=72: 267 s, 1.9 GB, extrapolated TV ≈ 0.18). The composition's "free Clifford" advantage is real but is constant/poly-factor engineering with substantial prior art; we position it accordingly.

1 Motivation

`clifft`'s state is split into (i) a *symbolic Clifford frame* evolving dormant axes for free, and (ii) a *dense active block*, a 2^k statevector expanded only for non-Clifford content. A static profiler (`research/stabrank_profiling`) established two facts that scope this work: k is built from magic (plain Clifford superposition never inflates it), and `clifft`'s measurement-reordering passes already shrink k aggressively. The bet: when the active block is *magic-sparse*, its stabilizer rank is $\ll 2^k$ and a low-rank residual extends `clifft`'s reach — at the cost of exactness.

2 Background and related work

CH-form and low-rank decompositions. A stabilizer state is stored as $|\phi\rangle = \omega\, U_C U_H |s\rangle$ (Bravyi–Browne–Calpin–Campbell–Gosset–Howard [1]): $O(n^2)$ bits per term, $O(n)$ Clifford gates, $O(n^2)$ Hadamard and single-amplitude queries. A magic state is a sum $|\psi\rangle = \sum_a c_a |\phi_a\rangle$ of χ terms; Cliffords preserve χ, each T doubles it, measurement can collapse it.

Related work (what is and is not new here). None of the individual ingredients is novel, and we position the composition against its prior art explicitly. "Cliffords paid once, exponential cost

only on the magic" is the architecture of Bravyi–Gosset [2] already; Qassim, Wallman and Gosset [3] recompile circuits into non-Clifford rotations times a single global Clifford precisely to speed up CH-form simulators; García–Markov [6] shared a stabilizer frame across superposition terms a decade ago; Pauli-based computation [7] absorbs all Cliffords into adaptive Pauli measurements on t magic qubits. The closest single work is Pashayan et al. [5]: gadgetize the T's, then *statically* compress the global Clifford and the outcome projector into an effective magic register before running a stabilizer-rank estimator. Stabilizer tensor networks with magic injection [8] pair a Clifford frame with an MPS residual; `quizx` [9] reaches the same "decompose only the magic" endpoint via ZX rewriting. What this prototype adds is the *online* form of the composition: a persistent global frame maintained gate-by-gate over a CH-form term sum, with `clifft`-style measurement-driven active-subspace collapse, supporting mid-circuit and adaptive measurement — plus the engineering observation (verified in qiskit-aer's source) that the flagship open-source CH-form simulator, `extended_stabilizer`, applies every Clifford per-term at $O(\chi)$ cost, with no frame. The advantage is a constant/poly factor, never an exponent, and we claim nothing more for it.

3 Method

3.1 Minimal-extent magic decomposition (the 0.228 exponent)

$T = \alpha I + \beta S$ with $\alpha = \frac{1}{2} + i\frac{\sqrt{2}-1}{2}$, $\beta = \bar{\alpha}$, both terms Clifford, $|\alpha| + |\beta| = 2^{0.114}$, so the stabilizer extent per T is $2^{0.228}$ — optimal within the extent framework, since extent is multiplicative for tensor products of single-qubit states [1]. Branching in the computational basis instead costs extent 2^t, which sparsification cannot repair; this basis choice is the crux.

3.2 Sparsification and single-shot magic sampling

Importance-sampling k terms from $p_a = |c_a|/\|c\|_1$ gives an unbiased $|\omega\rangle$ with $\mathbb{E}\||\psi\rangle - |\omega\rangle\|^2 = (\|c\|_1^2 - \|\psi\|^2)/k$; $k \sim 2^{0.228t}/\delta^2$ bounds the 2-norm error by δ. For *product* magic (IQP's $|T\rangle^{\otimes n}$) the branch strings factorize and we sample them directly — unbiased, no 2^t built, and no factor-t variance accumulation (streaming mid-circuit sparsification pays that factor; it remains implemented but is not the benchmarked path).

3.3 Norm estimation and normalization

$P(x) = |\langle x|\psi\rangle|^2/\|\psi\|^2$ needs $\|\psi\|^2$ without materializing 2^n; we implement the BGH estimator (Lemmas 2–4, $O(n^3)$ exponential sum, validated exactly). Two honesty points the earlier draft blurred. (i) The estimator's per-sample relative spread is ≈ 1.2, so L samples give $\approx 1.2/\sqrt{L}$ relative error *on every reported probability*; at the $L = 30$–80 used previously that is an 11–22% multiplicative error, the same order as the sparsification error — and the old "norm-free TV" metric cancelled it exactly. (ii) For the benchmark family — *unitary* product-magic circuits — $\|\psi\| = 1$ exactly and $\mathbb{E}\|\omega\|^2 = 1 + (\|c\|_1^2 - 1)/k$ is known analytically, so normalization needs no estimation at all (`samples=0` mode). We benchmark with analytic normalization and report the estimated-norm mode separately; *generic* circuits (projections, non-product magic) must pay the estimator, whose cost to keep the norm error subdominant scales as $\chi L \sim 2^{0.228t}/\delta^4$ — a real limitation, stated as such.

3.4 Error guarantee, stated precisely

Sparsification bounds the *2-norm* error $\|\psi - \omega\| \leq \delta$. For a single target that yields an *additive* amplitude error of typical size $\delta\, 2^{-n/2}$ — i.e. a $\sim \delta$ *relative* error on *typical* (anticoncentrated) IQP probabilities $P(x) \sim 2^{-n}$, which is what the measured TV $\approx 0.5\,\delta$ reflects. It is **not** a per-x relative-error guarantee: individual small probabilities can be off by more. Guaranteed relative error costs the exact-rank methods ($2^{0.396t}$ [4]).

3.5 Clifford-frame composition

As in the earlier draft: $|\text{state}\rangle = F(\sum_a |\phi_a\rangle)$ with F a global Clifford updated in $O(n)$ per gate; magic and measurement act on the terms conjugated through F at the optimal $\{I, S\}$ extent. Validated against the plain engine (including forced-outcome measurement) to 6×10^{-15}.

4 The honest baseline

The earlier draft timed `clifft.basis_probabilities` on the *unitary* IQP program, forcing peak_rank = n. That is not how a user computes $P(x)$: compiling the *measured* program and calling `record_probabilities` (exact forced replay) exploits `clifft`'s measurement-driven reduction. Measured facts (`validate_mitm.py`, review session):

- On the old benchmark's nearest-neighbour-CZ IQP the measured program compiles to peak_rank = 1 (lightcone sweep): `record_probabilities` returns the same 96 exact probabilities in 0.2 ms at n=26 — vs the 23 s the old benchmark charged `clifft` and the 1.3 s our backend took. **The old crossover was an artifact of the baseline.**
- On *dense* random IQP (CZ on each pair w.p. $\frac{1}{2}$, the standard hard ensemble) the measured program compiles to peak_rank = $n/2$: the honest exact baseline is $2^{n/2}$, not 2^n.
- Independently, a meet-in-the-middle (MitM) evaluator (`chform_cpp/mitm_iqp.cpp`) computes exact $P(x)$ for any T+CZ IQP in $O(n2^{n/2})$ time and $2^{n/2}$ memory — ground truth to $n \approx 62$ on a 36 GB machine, validated against dense (6×10^{-16}) and against `record_probabilities` (6×10^{-14} at n=24, 30).

So for this family, *everything exact costs* $2^{n/2}$, and hardness claims must be framed against that scale. (Multiplicative approximation remains GapP-hard for T-type IQP [10]; there is no poly-time escape, but n=72 is exactly checkable at $\sim 2^{36}$ work — by us.)

5 Implementation and validation

A Python prototype (`research/chform_backend`) is the reference; a bit-packed C++20 port (`research/chform_cpp`) is the performance implementation. This revision also fixed a latent bug: the C++ amplitude took a `uint64` target and shifted by qubit index — undefined behavior for $n > 64$ (the very frontier sizes previously reported, where it happened to work only for the all-zeros target). Amplitudes are now word-based (`amplitude_words`), and the multi-word ($n > 64$) paths are covered by new tests.

Table 1: Validation (max abs. error unless noted). Bold rows are new in this revision.

Component	Reference	Error
CH-form amplitude (Eq. 56)	hand-built states	3×10^{-17}
Full Clifford set	dense, 2000 circuits	8×10^{-15}
Single-qubit projection	dense, 2000 states	5×10^{-15}
Whole engine vs. `clifft`	`get_statevector`	2×10^{-16}
W=2 (n=80) embedded circuits	dense oracle, 900 runs	6×10^{-15}
Norm est. Lemma 4 (exp. sum)	brute force	0 (exact)
Norm est. Lemma 3	dense	5×10^{-16}
Norm est. Lemma 2	exact, L=6000	3.0%
W=2 (n=68) norm est., unit-norm state	exact (=1), L=600	4.8%
Frame engine vs. plain (incl. measurement)	forced outcomes	6×10^{-15}
MitM IQP evaluator	dense / `record_probabilities`	6×10^{-16} / 6×10^{-14}

6 Benchmarks

All contestants run single-threaded on an Apple M-series workstation (36 GB; `clifft` uses its scalar/NEON path — an x86 AVX-512 host would favor `clifft`); the MitM ground truth uses all cores but competes with no one. Scripts and the raw JSON are committed.

6.1 Extent, sparsification, free Cliffords

As before (unchanged, re-verified): measured L_1 growth per T is exactly $2^{0.114}$; sparsification error tracks the bound to $\sim 1\%$; the single-shot estimator is unbiased. On the measured magic-conveyor the frame composition performs **zero** per-term Clifford operations vs 473–15,411 for a measurement-aware pure stabilizer-rank simulator (w=4–8) — the constant-factor advantage of the composition, *cf.* [2, 3] for its static prior art.

6.2 The honest crossover

Dense random IQP, 96 uniform targets, exact ground truth by MitM (cross-checked against `record_probabilities` to 10^{-6} relative everywhere both run); `clifft` baseline = compile the measured program (peak rank $n/2$) + `record_probabilities`. Backend at budget $k = 2^{0.228n}/\delta^2$, analytic normalization (§3.3), timing includes build + normalization + all 96 amplitude evaluations. The error metric charges the full pipeline against the exact subset mass: $\mathrm{TV} = \frac{1}{2}\sum_x |\hat{P}(x) - P(x)| / \sum_x P(x)$. Means over 3 realizations ($n \le 48$; 1 above), Table 2 and Fig. 1:

The crossover is real but sits where the honest baseline puts it: $n \approx 34$ at $\mathrm{TV} \approx 0.17$, $n \approx 48$ at $\mathrm{TV} \approx 0.08$ — not $n \approx 22$. `clifft`'s exact path grows as $2^{n/2}$ on the nose ($\times 4.6$ per $+4$ qubits) and exits at $n \approx 58$–62 on memory; the backend's growth is $2^{0.228n}$ ($\times 1.9$ per $+4$).

6.3 The accuracy dial

At n=44, 256 targets, 4 realizations, analytic normalization (Fig. 1, right): TV = 0.144(4), 0.101(3), 0.0500(6), 0.02 at $\delta = 0.30, 0.20, 0.10, 0.05$ — i.e. $\mathrm{TV} = (0.50 \pm 0.01)\,\delta$, with runtime $\propto k \propto 1/\delta^2$ (2.0 → 73.9 s). The earlier draft's "TV $\approx 0.47\,\delta$" was approximately right, *but only because its metric excluded the norm error*: re-running the same budgets with the L=30 estimated norm (the old setting)

Table 2: Honest head-to-head (seconds for 96 exact/approximate $P(x)$).

n	k_{meas}	`clifft` exact	ours δ=0.4	TV	ours δ=0.15	TV
24	12	**0.003**	0.09	0.070	0.08	0.070
32	16	**0.06**	0.15	0.140	0.50	0.073
36	18	0.27	**0.18**	0.192	1.22	0.081
40	20	0.77	**0.41**	0.175	2.94	0.079
44	22	4.1	**1.0**	0.194	7.4	0.076
48	24	17.3	**2.4**	0.197	**16.9**	0.082
52	26	92.9	**5.5**	0.186	**39.4**	0.074
56	28	398	**12.1**	0.165	**86.2**	0.078
60	30	≥16 GB, skipped	**27.7**	0.185	**196**	0.086

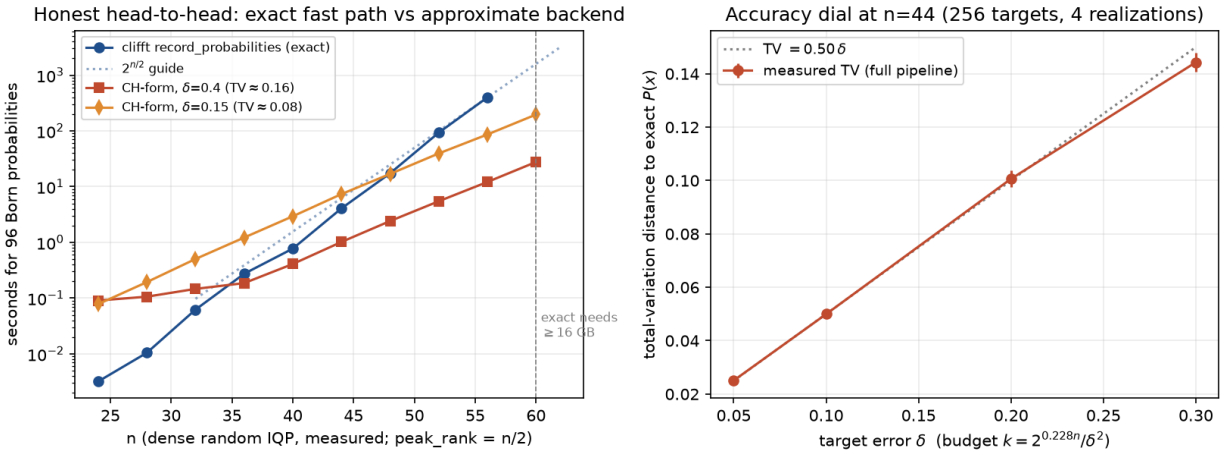


Figure 1: *Left:* seconds for 96 Born probabilities on measured dense IQP: `clifft`'s exact forced-replay path vs the CH-form backend at two accuracy settings; the dotted guide is $2^{n/2}$. *Right:* the accuracy dial at n=44: full-pipeline TV against exact $P(x)$ vs the target δ, with the TV = 0.50 δ line.

degrades same-budget TV by up to $\sim 2\times$ on unlucky draws (0.07–0.32 observed vs 0.07–0.21 analytic), exactly the $1.2/\sqrt{30} \approx 22\%$ multiplicative noise of §3.3. The clean law holds for the full pipeline only where normalization is analytic (unitary product magic) or the norm budget is scaled up ($\chi L \sim 1/\delta^4$).

6.4 The frontier, honestly framed

Dense IQP, δ=0.4, single-shot build + one amplitude (`bench_iqp.cpp`). The exact column is what *any* known exact method pays on this family ($2^{n/2}$: `clifft`-measured or MitM); the old draft's "2^n / 64 ZB" contrast overstated the gap by a factor $2^{n/2}$.

Accuracy at the frontier: exact ground truth exists to $n \approx 62$ (MitM), where the measured TV still follows the dial (0.086 at n=60, δ=0.15). At n=66–72 **no exact check exists** (that is the point); accuracy there rests on (i) the δ-law holding at every checkable size, (ii) the analytic normalization being exact for this family, and (iii) the n>64 code paths now being tested (W=2 rows of Table 1) — an extrapolation, stated as such, not a measurement.

Table 3: Past-exact frontier (dense IQP, δ=0.4).

n	χ	build	CH-form mem	exact $2^{n/2}$ mem
40	3 477	0.4 s	3.6 MB	16 MB (feasible)
50	16 889	3.0 s	22 MB	512 MB (feasible)
60	82 029	25 s	128 MB	16 GB (marginal)
66	211 728	82 s	683 MB	137 GB (infeasible)
72	546 497	267 s	1.9 GB	1.1 TB (infeasible)

6.5 Beyond product magic: gadgetized general circuits

Every scaled result above uses the single-shot sampler, which needs all magic in one product layer. The generalization to *arbitrary* Clifford+T circuits (`gadgetize.py`, C++ arm `bench_gadget.cpp`): teleport each in-line T onto its own ancilla and force the gadget outcome to 0 — since the gadget's classically-controlled correction makes all outcome branches equivalent, each forced-0 gadget contributes exactly $T/\sqrt{2}$, so $P(x) = 2^t\,|\langle x, 0^t|C_{\text{gadget}}(|0^n\rangle\otimes|T\rangle^{\otimes t})|^2$ with C_{gadget} entirely Clifford. All magic is again one product layer: the single-shot sampler applies with **no streaming t-factor**, and the normalization stays *analytic* — the norm-estimation bottleneck of §3.3 never enters. Validated: the $7T$ CCZ network and exact gadgetized = dense = `clifft` to 10^{-14} on random interleaved circuits; sparsified TV scales with δ; the streaming `sparsify` path is now also exercised (unbiased), closing that coverage gap.

At scale, the hidden-shift family (bent-function construction, CCZ oracles, 14 T's per CCZ) provides *built-in exact ground truth*: the output is the shift s deterministically, so $P(s) = 1$ at any size. Measured at $\delta = 0.3$: n=16/24/32/40 (t = 28–56, up to 96 total qubits) give $\hat{P}(s) =$ 0.99/0.94/0.90/0.92 in 0.1–8 s, with $\hat{P}(x \neq s) = 0$ — interleaved magic handled at the product-magic price.

Two honest observations. First, `clifft`'s measurement-driven reduction compiles *both* tested general families to tiny blocks (hidden shift: peak_rank ≤ 10; our random interleaved Clifford+T: ≤ 6) and answers exactly in microseconds — these families validate the backend's generality, they do not exhibit a regime where it wins. Second, this sharpens into a **compile-time decision rule**: the backend's exponent is $0.228\,t$, `clifft`'s is peak_rank, and both are known *before executing anything* — a hybrid dispatcher can route each program to the cheaper engine ($0.228\,t <$ peak_rank: backend; otherwise `clifft`). Dense IQP ($t = n$, peak_rank $= n/2$) sits on the backend's side, as §6.2 measured; magic-dense or reduction-friendly circuits sit on `clifft`'s. The rule must use the *live* T-count after `clifft`'s HIR optimization, not the raw one: measured on the random interleaved family, $t_{\text{raw}} = 32/40/48$ compiles down to $t_{\text{live}} = 18/28/28$ — using t_{raw} biases the dispatch against the backend.

6.6 qiskit-aer `extended_stabilizer`

Now reproduced by a committed script (`bench_aer.py`; aer 0.17.2, settings inline — the old draft's numbers were unauditable). Sampling (nn-IQP, 2000 shots, mixing 50): `clifft` samples exactly at 1.8–4.4×10^6 shots/s (peak_rank = 1) vs aer's approximate 2721 $\to$ 44 shots/s over $n = 10 \to 28$ — ratios 1630× to 40,000×. Strong simulation (dense IQP n=24, 96 uniform targets, true $P \sim 5.6 \times 10^{-8}$): `clifft` answers exactly in 3.5 ms; aer's only route is shot frequency, and 30,000 shots (455 s) hit 0/96 targets ($\sim 2 \times 10^7$ shots needed per expected hit). The three-way split stands: `clifft` owns exact work while $2^{n/2}$ fits; this backend owns approximate $P(x)$ beyond;

`extended_stabilizer` (sampling-only, maintenance-frozen upstream) is dominated on both tasks.

6.7 External baselines: QuiZX (and Quokka#)

QuiZX [9] (v0.3.0, PyPI wheel) computes each Born probability *exactly*, one ZX-simplify+decompose run per target, validated here against MitM to 10^{-6} relative on every instance. On the SAME dense-IQP instances and targets as §6.2 (`bench_external.py`), its per-amplitude cost grows as the exact-rank rate $2^{0.396n}$ (measured term counts $301 \to 1.9 \times 10^6$ over n=24 → 56, ×9.0 per 8 qubits: ZX simplification does *not* compress dense T+CZ IQP), giving for 96 targets: 0.5 s (n=24), 251 s (n=48), 2188 s (n=56) — vs this backend's 0.3/17/89 s at TV ≈ 0.08. Two honest qualifiers. (1) QuiZX answers a *harder* question (exact vs additive-approximate); the comparison shows complementary regimes, not dominance. (2) Our own MitM evaluator is the strongest *exact* method on this family — 0.4/4.7/84.5 s for the same queries, i.e. it *ties the approximate backend through* $n \approx 56$ — so on pure T+CZ IQP the approximate backend's real niche over *all* known exact methods only opens past the $2^{n/2}$ memory wall ($n \gtrsim 60$), or on circuit families without MitM structure. Quokka# [11] (with its GPMC model counter, built locally) was run on the same instances: exact per-amplitude, 3.1 s at n=24 (correct to 5×10^{-15}) but *timeout* (> 300 s/amplitude) already at n=32 — the dense CZ graph defeats the CNF encoding far below the tool's published sparse-circuit range. On this family the external exact ranking is QuiZX ≫ Quokka#, with our MitM evaluator fastest of all three.

6.8 The adaptive workload: where the composition pays off — and where it does not

The composition's defensible novelty is *online* operation: mid-circuit measurement with classical feedforward. We benchmark an adaptive family (`bench_adaptive.py`): a width-w register, R=4 rounds of H-layer, feedforward-X conditioned on the previous round's outcomes, a $T^{\pm}$ layer, D global Clifford scrambling layers applied *while* $\chi = 2^w$, a CZ chain, H-layer, then measurement of all w qubits. Trajectories are sampled once (dense reference), then *replayed* by both engines — identical circuits, identical outcomes, identical CH-form term stores and measurement code; the only difference is who pays for Cliffords (plain: per-term, the verified architecture of Aer's `extended_stabilizer`; frame: $O(n)$ absorption). All engine variants agree to 3×10^{-16}.

Three findings, two of them not in the composition's favor:

1. **The incumbent cannot enter.** qiskit-aer `extended_stabilizer` *rejects* the workload outright at every size (dynamic-circuit `mark`/`jump` instructions unsupported). Online adaptive simulation at the CH-form representation is territory the flagship open-source implementation does not serve at all.

2. **Free Cliffords are real but not free lunch.** At w=12, D=4: the plain engine spends 68 s applying Cliffords per-term where the frame spends 3 ms — but the frame engine *loses overall* (217 s vs 77 s), because frame-conjugated magic and measurement act through $P' = F^{-1} Z_q F$ of weight $\sim w$, costing $O(w)$ per-term gate applications per T and per measurement. The frame does not eliminate per-term work; it *moves* it from Cliffords onto magic and measurement.

3. **The measured phase boundary.** Sweeping D at w=8: frame time is *flat* (6.9–7.9 s from D=4 to 96) while plain grows linearly (3.1 → 65.9 s); the crossover sits at $D^* \approx 10 \approx 1.2w$, and at D=96 the frame is 8.4× faster and the gap grows linearly in D. **The online composition**

pays off precisely when the adaptive circuit is Clifford-dominated ($\gtrsim w$ Clifford layers per magic/measure round — syndrome-extraction-like traffic), and costs $\sim 2\times$ overhead when it is not.

Natural workloads land on the winning side. The synthetic sweep leaves one question: do *recognizable* protocols sit above or below D^*? Two natural adaptive families (`bench_natural.py`; trajectories sampled once on a dense reference, replayed identically by both engines; all variants agree to 10^{-16}): *(i) cultivation-style patches* — a data register accumulates injected T's (never measured away) while every round runs syndrome extraction (ancilla CX fans, measurements, classically-controlled corrections, ancilla reuse) plus L layers of code-preserving logical Clifford traffic (deformation/surgery); *(ii) teleported-T injection* — the textbook adaptive magic gadget (ancilla $|T\rangle$, CX, measure, classically-controlled S correction), the minimal-Clifford contrast point. Measured (8 data qubits, 7 checks, up to 10 injected T's, χ up to $\sim 10^3$):

Table 4: Natural adaptive workloads: plain (per-term Cliffords) vs frame, identical trajectories.

workload	Cliffords/round	frame speedup
bare syndrome extraction (L=0)	$\sim$14	0.8–1.0× (at the boundary)
+ 1 logical-traffic layer	$\sim$32	3.7–6.7×
+ 4 layers	$\sim$68	7–16×
teleported-T gadget	$\sim$4/gadget	1.7–2.8×

Bare syndrome extraction sits *exactly at* the measured boundary, and any logical Clifford traffic at all pushes the composition into clear wins — the plain engine's per-term Clifford work is the entire gap (e.g. 11.8 s of a 12.0 s round set vs the frame's 7 ms). Even the minimal teleported gadget benefits, because the frame absorbs its H/CX while χ is large. One physical subtlety the experiment surfaced: the logical traffic must be *code-preserving* (diagonal + CNOT-type; no bare H on data), else the syndrome round measures the accumulated magic away — true of the toy and of the real protocols it models.

7 The conceptual frame: one compiler, two executors

The results above assemble into a single architecture, and it is worth stating plainly, because it also settles what this backend *is* relative to `clifft`.

What comes from where. The backend borrows `clifft`'s two architectural ideas — the global symbolic Clifford frame (absorb Cliffords in $O(n)$, keep the non-Clifford core small) and measurement-driven collapse of that core — and re-instantiates them over a different core: BGH's low-rank CH-form sum instead of a dense 2^k array. The simulation mathematics (CH-form, minimal-extent decomposition, sparsification, norm estimation, gadgetization) is BGH/Bravyi–Gosset throughout; none of `clifft`'s runtime is used. `clifft` appears in three supporting roles: validation oracle (every component is checked against it to machine precision), the honest benchmark opponent, and — the role that matters going forward — the *compiler front-end*.

Compile once, dispatch per program. Both executors' cost exponents are known from the compiled artifact before anything runs: `clifft`'s dense engine pays $2^{\text{peak\_rank}}$ (the k-schedule is

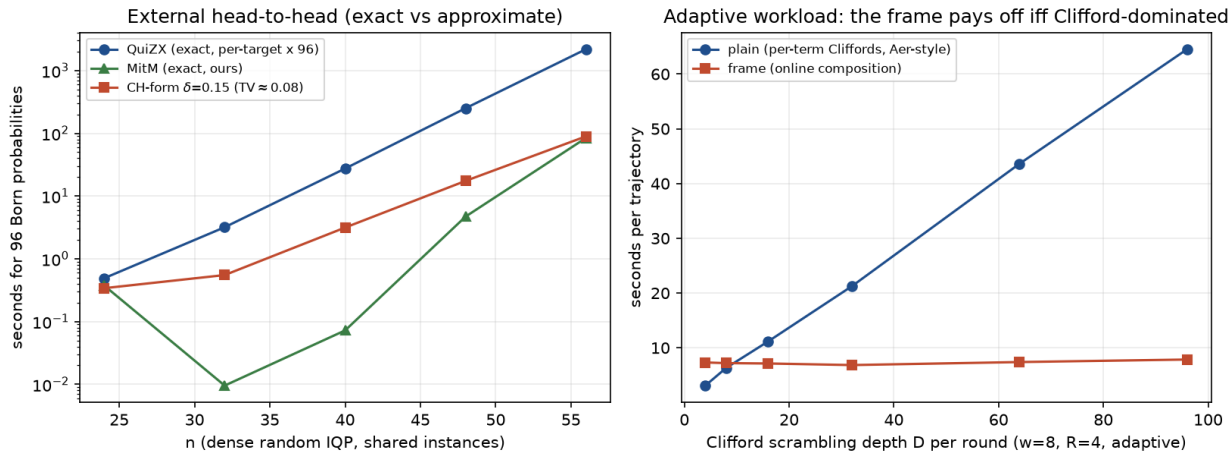
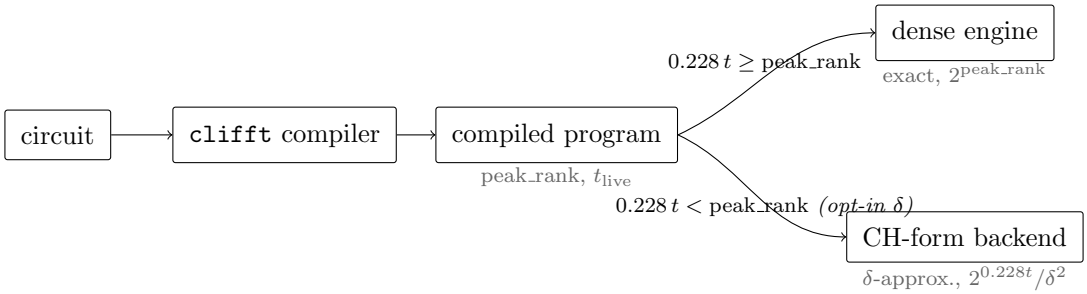


Figure 2: *Left:* external head-to-head on shared dense-IQP instances: QuiZX (exact, per-target) vs the MitM evaluator (exact) vs this backend (approximate, TV ≈ 0.08), seconds for 96 probabilities. *Right:* the adaptive-workload phase boundary at w=8: plain (per-term Cliffords) grows linearly in Clifford depth D; the frame composition is flat; crossover at $D^* \approx 1.2w$.

compile-time static), and the CH-form backend pays $\sim 2^{0.228\,t}/\delta^2$ (the live T-count survives the same compilation). Dispatch is a one-line comparison:



This is no longer only a proposal: the backend now **consumes `clifft`'s optimized HIR directly** (`hir_bridge.py`, using the exposed `parse`→`trace`→`HirPassManager` pipeline). The optimized HIR turns out to be the ideal input, for a reason worth stating: *it contains no Clifford gates at all* — the compiler absorbs every Clifford into the Pauli strings of the surviving ops, i.e. it performs the frame absorption *statically and exactly*. The remaining ops are precisely the backend's two explicit-Pauli primitives (T-rotations about arbitrary Paulis; projective Pauli measurements), so the bridge is thin. Validated against `clifft` to 2×10^{-13} over 400 fuzzed circuits plus the benchmark families; in every HIR-driven run the backend performs *zero* Clifford gate applications, χ tracks $2^{t_{\text{live}}}$ (often less: the hoisted measurements collapse terms mid-run — $\chi_{\text{peak}} = 8$ where $2^{t_{\text{live}}} = 64$ on one family), and t_{live} is roughly half of t_{raw} on random circuits. Consequently, for compile-known circuits the compiler *supersedes* the online frame of §6.8; the online frame remains the mechanism for dynamic/adaptive circuits the compiler cannot see through. The C++ scale arm still consumes raw circuits (consuming HIR there needs a simultaneous reduction of the terminal Pauli measurements — a symplectic elimination — noted as future work).

Three design consequences. *(i) Both executors want the same compiled stream.* Every compiler improvement (T-count reduction, fusion, measurement reordering) lowers both engines' costs —

now demonstrated, not assumed: the Python reference consumes the optimized stream; only the C++ arm's numbers remain conservative. *(ii) The dispatcher must respect the exactness contract.* The backend is a different guarantee (δ-approximate, additive/typical — §3.4), so it is an opt-in path (an explicit δ), never a silent substitute: when the user demands exactness past the memory wall, the honest answer is "infeasible". *(iii) The finer-grained, per-episode rule is implemented* (`dispatch.py`). Compiled programs have episode structure (k rising and collapsing to 0); the static profiler supplies m_{blk} (peak T-count inside any single episode) and the episode count R, and the dispatcher weighs a third strategy: re-sparsify at every episode boundary, cost $\sim R^2\, 2^{0.228\, m_{\text{blk}}}/\delta^2$ (errors compound over R boundaries, so $\delta_{\text{episode}} = \delta/\sqrt{R}$). Since $m_{\text{blk}} \le t_{\text{live}}$, this only moves circuits toward the backend — decisively for computations that *recycle* bounded live magic: on a feedforward conveyor with 12 rounds of 128 T's, the dense engine is infeasible (k=49), the global backend absurd ($0.228 \cdot 1536 \approx 350$), and the episodic strategy ($\log_2$ cost ≈ 42) is the only one that runs — the dispatcher selects it automatically. The premise is validated in exact mode ($\chi_{\text{peak}} \approx 2^{m_{\text{blk}}} \ll 2^{t_{\text{live}}}$ on a conveyor via the HIR bridge, which now also replays the feedforward's conditional Paulis; P matches `clifft` to 4×10^{-17}); the *approximate* episodic execution (per-episode budgets $\delta/\sqrt{R}$, mid-circuit norm tracking) remains future work, so its R^2 prefactor is a projected cost, not a measured one.

In one sentence: `clifft`'s architecture is the blueprint, BGH's mathematics is the material, and `clifft` itself is the compiler front-end that decides — per program, before execution — whether the dense exact engine or the low-rank approximate backend runs.

8 Limitations

- **Approximate**, with the guarantee of §3.4 (additive/typical, not per-x relative), and `clifft`'s exactness forfeited.
- **The exact competition is $2^{n/2}$, not 2^n**, on this family; the backend's genuinely-unique regime starts near $n \approx 62$, and its speed advantage at moderate accuracy starts near $n \approx 34$–48.
- **Norm estimation** is the pipeline's accuracy-cost bottleneck for generic circuits ($\chi L \sim 1/\delta^4$); the clean results here lean on analytic normalization, valid for unitary product magic only.
- **The composition's value is measured and two-sided, and natural protocols land on the winning side** (§6.8, Table 4): the frame wins iff the circuit is Clifford-dominated while χ is large ($D^* \approx 1.2w$ on the synthetic sweep); bare syndrome extraction sits at the boundary, and cultivation-style workloads with any logical Clifford traffic win 4–16× (teleported-T gadgets $\sim 2\times$). The conjugation of magic and measurement through the frame remains the price of free Cliffords. Prior art for the static forms remains [2, 3, 6, 5].
- **Prototype gaps:** the Python engine's sampling measurement path and canonical dedup materialize 2^n (validation-scale; the forced-replay path, `measure_z_forced_fast` + `collapse_to_rank1`, is non-materialising); the C++ engine has no measurement path (build-and-query only; the gadgetized route, §6.5, removes the need for mid-circuit measurement in strong simulation). The adaptive wall-clock numbers are Python-vs-Python (fair internally, absolute constants not comparable to C++ tools).
- `quEStab` (ACM ICS 2026, multi-GPU extended stabilizer) still needs a manual literature check (paywalled, no preprint).

9 Conclusion

The validated machinery stands, and so does a real — but honestly smaller — niche. Against `clifft`'s true exact fast path on dense IQP, the CH-form backend wins from $n \approx 34$ (coarse) / $n \approx 48$ (TV ≈ 0.08), and past $n \approx 62$ it is the only method of the three (dense block, forced replay, MitM) that runs at all: n=72 in 267 s and 1.9 GB where anything exact needs $\sim$1 TB. The accuracy dial is clean and predictable (TV $= 0.50\,\delta$) once normalization is handled honestly. What this note deliberately no longer claims: a crossover at $n \approx 22$ (baseline artifact), 2^n-scale frontier contrasts ($2^{n/2}$ is the honest exact scale), sub-10% accuracy at budgets whose norm error exceeded that (metric artifact), and novelty for free Cliffords as an idea (prior art).

The follow-up experiments are now done, each with a two-sided answer. *External baselines* (§6.7): the backend beats QuiZX's exact per-target cost by growing margins (2188 s vs 89 s at n=56 for 96 targets, exact vs TV $\approx$ 0.08), but our own exact MitM evaluator ties the backend through $n \approx 56$ — on pure T+CZ IQP the approximate niche opens only past the $2^{n/2}$ wall or off the IQP structure. *The adaptive workload* (§6.8): the incumbent CH-form implementation cannot run adaptive circuits at all; the online frame composition is flat in Clifford depth where per-term simulation grows linearly, winning beyond a measured phase boundary $D^* \approx 1.2w$ — and losing $\sim 2\times$ below it, because frame conjugation shifts per-term cost onto magic and measurement. Crucially, *natural* protocols land on the winning side (Table 4): cultivation-style patches with logical Clifford traffic win 4–16×, teleported-T gadgets $\sim 2\times$, and only bare syndrome extraction sits at the boundary. The composition is a real, now-quantified advantage precisely for the fault-tolerance-shaped circuits it was designed for. Architecturally, the pieces assemble into one compiler with two executors chosen at compile time (§7).

References

[1] S. Bravyi, D. Browne, P. Calpin, E. Campbell, D. Gosset, M. Howard, *Simulation of quantum circuits by low-rank stabilizer decompositions*, Quantum **3**, 181 (2019), arXiv:1808.00128.

[2] S. Bravyi, D. Gosset, *Improved classical simulation of quantum circuits dominated by Clifford gates*, Phys. Rev. Lett. **116**, 250501 (2016), arXiv:1601.07601.

[3] H. Qassim, J. J. Wallman, D. Gosset, *Clifford recompilation for faster classical simulation of quantum circuits*, Quantum **3**, 170 (2019), arXiv:1902.02359.

[4] H. Qassim, H. Pashayan, D. Gosset, *Improved upper bounds on the stabilizer rank of magic states*, Quantum **5**, 606 (2021), arXiv:2106.07740.

[5] H. Pashayan, O. Reardon-Smith, K. Korzekwa, S. D. Bartlett, *Fast estimation of outcome probabilities for quantum circuits*, PRX Quantum **3**, 020361 (2022), arXiv:2101.12223.

[6] H. J. García, I. L. Markov, *Simulation of quantum circuits via stabilizer frames*, IEEE Trans. Computers **64**, 2323 (2015), arXiv:1712.03554.

[7] F. C. R. Peres, E. F. Galvão, *Quantum circuit compilation and hybrid computation using Pauli-based computation*, Quantum **7**, 1126 (2023), arXiv:2203.01789.

[8] A. C. Nakhl et al., *Stabilizer tensor networks with magic state injection*, Phys. Rev. Lett. **134**, 190602 (2025), arXiv:2411.12482.

[9] A. Kissinger, J. van de Wetering, *Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions*, Quantum Sci. Technol. **7**, 044001 (2022), arXiv:2109.01076.

[10] M. J. Bremner, A. Montanaro, D. J. Shepherd, *Average-case complexity versus approximate simulation of commuting quantum computations*, Phys. Rev. Lett. **117**, 080501 (2016), arXiv:1504.07999.

[11] J. Mei, M. Bonsangue, A. Laarman, *Simulating quantum circuits by model counting*, CAV 2024, arXiv:2403.07197 (Quokka#).